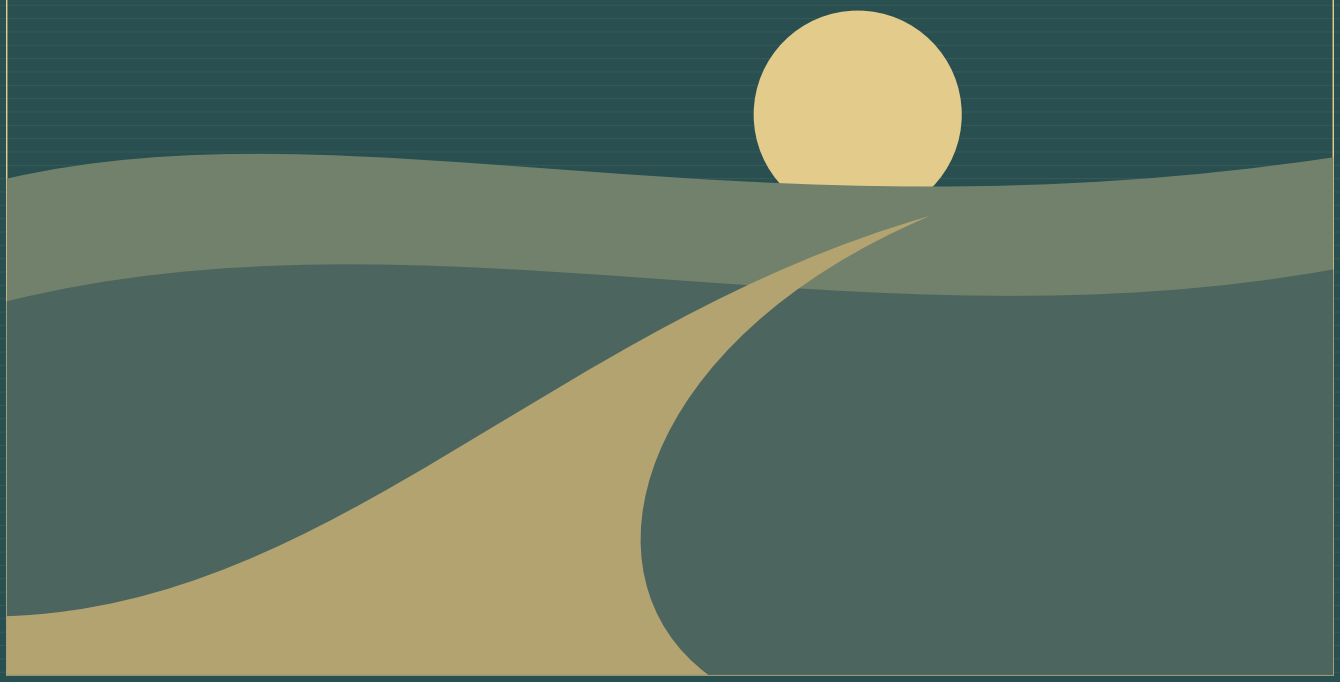
A PRACTICAL FIELD GUIDE

The LIVING REPOSITORY

Project memory with Codex

READ · SHAPE · RETURN



MAKE CONTEXT VISIBLE. KEEP WORK ALIGNED.

1. Give context a home

A chat helps you think through a project, but details are easy to lose when you start another. A small folder of Markdown files keeps current work, decisions, and next steps. You can read and correct them, then ask Codex to use them when you return.

Markdown files end in `.md`. They are plain text with headings and lists, easy to read, edit, and move between tools. The starter has a map, instructions for Codex, and three project files:

```
my-ai-workspace/
+-- README.md
+-- AGENTS.md
+-- projects/
    +-- my-project/
        +-- state.md
        +-- decisions.md
        +-- tasks.md
```

Download this repository from GitHub with **Code → Download ZIP**. Extract the ZIP, open the extracted folder, and copy the entire `my-ai-workspace` folder to a place you keep your work. Start with the files it contains; add more only when a real project needs them. Keep a backup copy of the folder before making your first changes. If an edit goes wrong, use it to restore the original text.

The files carry memory between chats. Ask Codex to read what it needs when you begin. Save settled facts and choices, not every conversation: current facts, unknowns, and next step in `state.md`; choices and reasons in `decisions.md`; open actions in `tasks.md`.

2. Work from the folder

Use the Codex desktop app, installed and signed in. Create a local project for your copied folder. If you already have a project, open its project menu, choose **Edit project**, then **Add folder**. Set your workspace folder as the primary folder so new chats use it. The official project instructions explain project folders. Opening files in a browser does not give Codex access to your local copy.

Before editing, ask Codex to confirm the folder and read its guidance and map. For example:

```
Show the full path of the folder you are working in. Read AGENTS.md and
README.md, then the three files in projects/my-project/. Summarize them.
Do not change anything yet.
```

If the files are missing, reconnect the folder. Otherwise, try one small task:

```
Rename projects/my-project/ to projects/skill-share/. Update the README links.
Plan a skill-share for 12 people, with a $60 total budget and venue unknown.
Save its purpose and facts in state.md, and next actions in tasks.md.
Keep unconfirmed details marked unknown.
```

Ask Codex to open the actual saved `state.md` in the app. Read it, then try a change:

```
The total budget is now $40 to keep the session affordable.
Update the skill-share state and tasks. Record the change and reason in
its decisions.md. Keep the venue unknown.
```

Open the saved files and correct anything inaccurate. The current plan should use $40; the old $60 belongs only in the decision history.

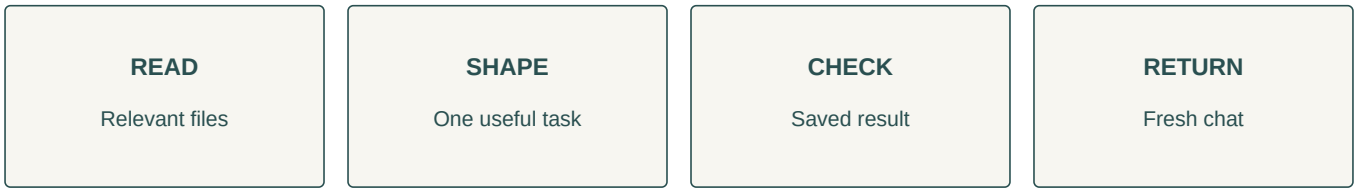
3. Save your next starting point

At the end of a chat, ask Codex to leave a short handoff in `projects/skill-share/state.md`: current facts, unknowns, and the next useful step. Keep open actions in `projects/skill-share/tasks.md` and choices with reasons in `projects/skill-share/decisions.md`. Open the actual saved files and check them.

In a fresh chat in the same local project, ask Codex to read `AGENTS.md`, `README.md`, and those three project files. Ask it to report the headcount, current budget, venue status, and next step. It should find 12 people, $40, and an unknown venue without needing you to retell the story.

The worked example shows the saved result.

Keep only context that is useful, and leave sensitive information out. Local files stay in your folder, but an online AI may process the contents of files it reads.



Read less old chat. Keep a clear next step.

Guide and starter: github.com/xichia/persistent-llm-memory-system